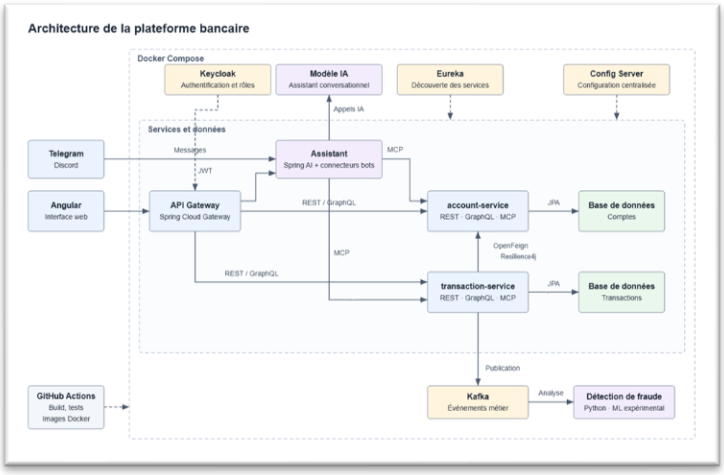


ADRIA TEST PFE

Prof. YOUSSFI

ARCHITECTURE MICRO-SERVICES, IA ET DEVELOPPEMENT FULL-STACK

NAZIH Youssef

GLSID 3 – 2026/2027

Contents

1. Préparation du projet 3
2. Conception et architecture 3
   2.1. Diagramme de classes 3
   2.2. Schéma d'architecture de projet 4
3. Développement des micro-services fonctionnels 4
   3.1. Account-Service 4
   3.2. Transaction-Service 5
   3.3. OpenFeign et Resilience4J 6
      3.3.1. OpenFeign 6
      3.3.2. Resilience4J 6
4. Les micro-services techniques 8
   4.1. Spring Cloud Gateway 8
   4.2. Eureka Discovery Service 8
   4.3. Config Service 9
5. ChatBot Agentique 9
6. Client Flutter 10

Table de figures

Figure 1: Repository GitHub privé 3
Figure 2: Diagramme de classes 3
Figure 3: Schéma d'architecture de projet 4
Figure 4: Requete GET Accounts 5
Figure 5: Requete GET Transactions 5
Figure 6: Informations accounts visible avec OpenFeign 6
Figure 7: Consultation des transactions avec les détails du compte 7
Figure 8: Fallback : transactions accessibles malgré la panne 7
Figure 9: Création refusée pendant la panne : HTTP 503 7
Figure 10: Spring Cloud Gateway 8
Figure 11: Eureka dashboard 8
Figure 12: Config Service 9

Figure 13: Discord bot 9
Figure 14: Client Flutter 10

1. Préparation du projet

La préparation du projet a commencé par la création d'un dépôt GitHub privé conforme aux consignes, puis par son clonage sur mon ordinateur portable. Le projet a ensuite été initialisé avec la création du microservice Spring Boot Account-Service.



Figure 1: Repository GitHub privé

2. Conception et architecture

2.1. Diagramme de classes

Ce diagramme présente les classes Account et Transaction ainsi que leurs services. Un compte peut être associé à plusieurs transactions, et les services permettent de consulter les comptes, de mettre à jour les soldes, d'enregistrer les transactions et d'effectuer des virements.

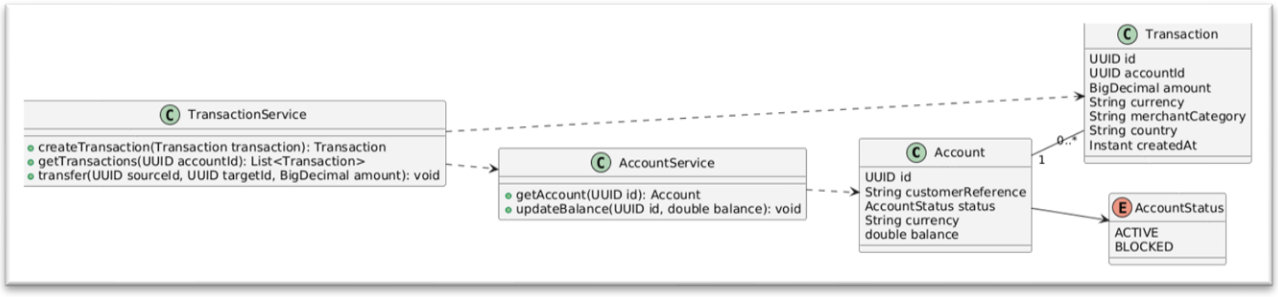


Figure 2: Diagramme de classes

2.2. Schéma d'architecture de projet

Le projet repose sur deux microservices, account-service pour la gestion des comptes et transaction-service pour les transactions. Chacun possède sa propre base de données et expose ses fonctionnalités via REST, GraphQL et MCP. Leur communication utilise OpenFeign avec Resilience4j pour gérer les pannes.

L'accès web passe par une gateway, avec une authentification assurée par Keycloak. Un assistant Spring AI permet également d'interroger les données depuis le web, Telegram et Discord. Kafka transmet les événements au module de détection de fraude, tandis que Docker Compose permet l'exécution locale des composants.

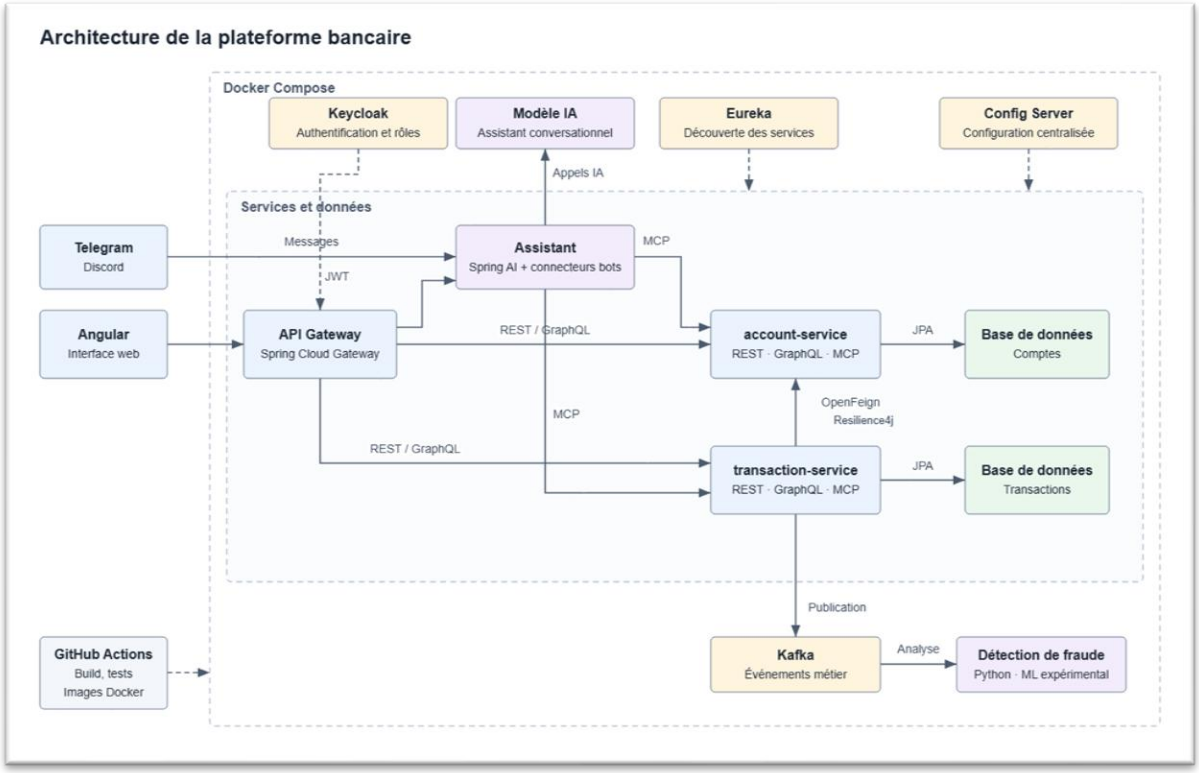


Figure 3: Schéma d'architecture de projet

3. Développement des micro-services fonctionnels

3.1. Account-Service

Le microservice Account-Service comprend une entité Account, un repository étendant JpaRepository et un contrôleur REST proposant trois méthodes : findAll, findById et save. Les UUID sont attribués manuellement, sans utiliser @GeneratedValue, afin de connaître les identifiants à l'avance et de faciliter les tests et le débogage.

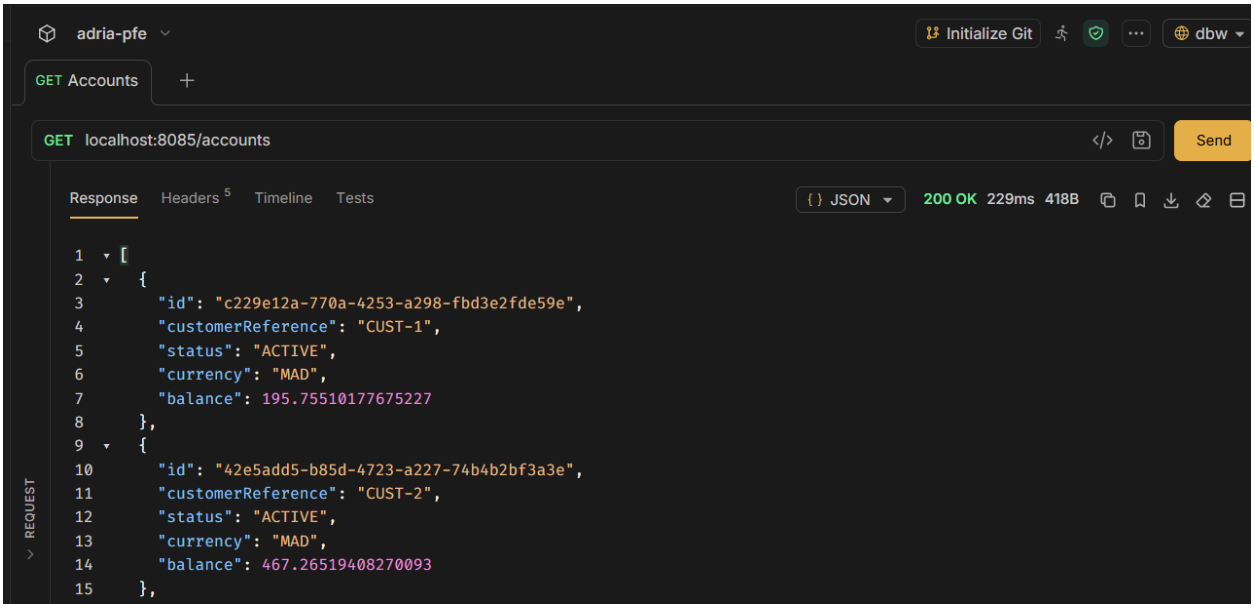


Figure 4: Requete GET Accounts

3.2. Transaction-Service

Le microservice Transaction-Service reprend la même structure qu'Account-Service, avec une entité Transaction, un repository étendant JpaRepository et un contrôleur REST. Il ne dispose cependant pas encore d'un accès aux informations des comptes. Cette communication sera mise en place dans le chapitre suivant à l'aide d'OpenFeign.

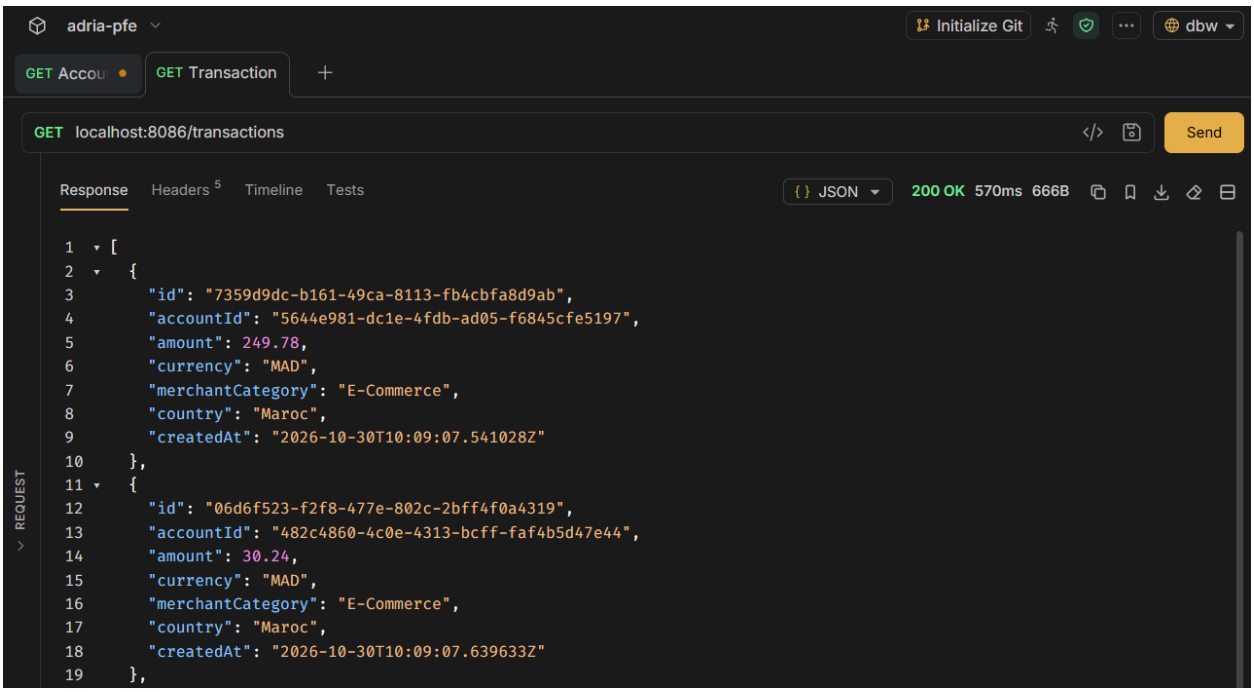


Figure 5: Requete GET Transactions

3.3. OpenFeign et Resilience4J

3.3.1. OpenFeign

OpenFeign simplifie la communication entre microservices grâce à une interface Java annotée avec @FeignClient. Les méthodes de cette interface décrivent les routes REST à appeler. Spring génère ensuite le client HTTP et transforme les réponses reçues en objets Java.

Dans Transaction-Service, nous avons créé l'interface AccountRestClient pour consulter Account-Service à partir de l'identifiant d'un compte. Ce client est injecté dans TransactionService pour compléter les transactions avec les informations du compte lors des consultations. Avant l'enregistrement, ces informations permettent de vérifier que le compte est accessible, actif et utilise la même devise que la transaction. Un fallback est également déclaré pour gérer l'indisponibilité d'Account-Service.

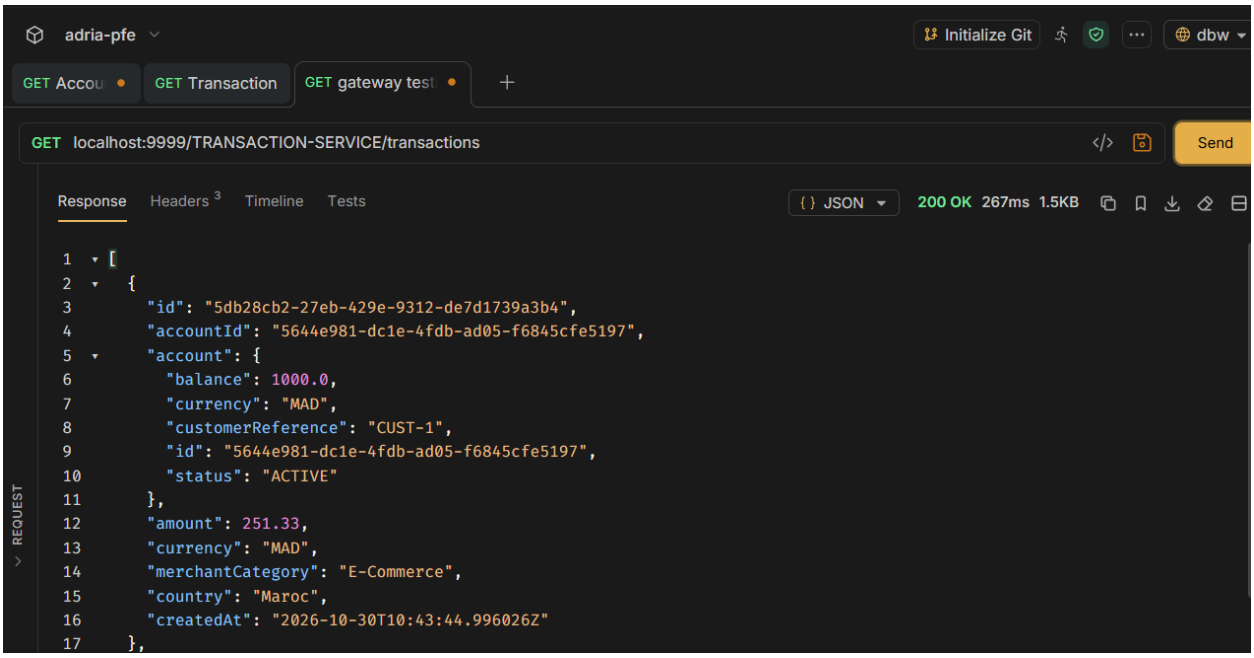


Figure 6: Informations accounts visible avec OpenFeign

3.3.2. Resilience4J

La tolérance aux pannes est assurée par Resilience4J, intégré au client OpenFeign. Lorsque account-service est indisponible, un fallback permet de consulter les transactions avec account: null. La création d'une transaction est refusée avec le statut HTTP 503, car le compte ne peut pas être vérifié. Après rétablissement du service, les informations des comptes sont de nouveau accessibles.



Figure 7: Consultation des transactions avec les détails du compte

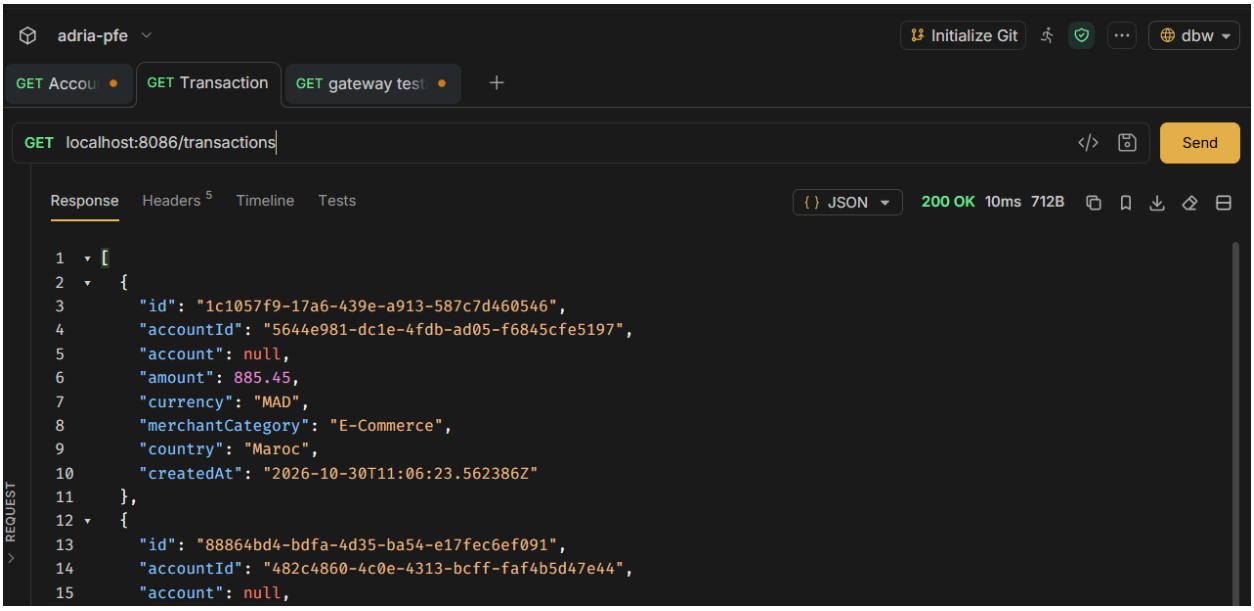


Figure 8: Fallback : transactions accessibles malgré la panne

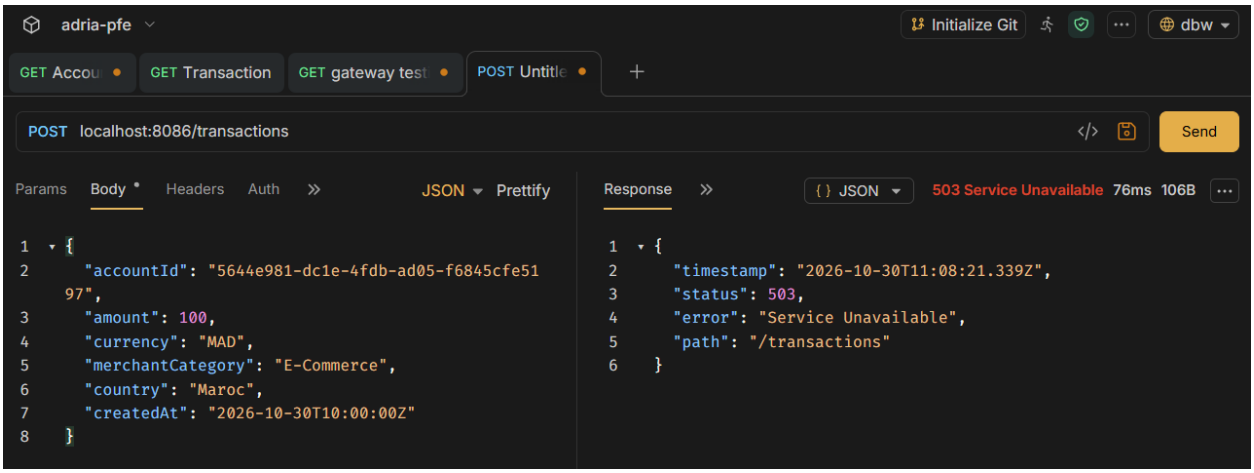


Figure 9: Création refusée pendant la panne : HTTP 503

4. Les micro-services techniques

4.1. Spring Cloud Gateway

Spring Cloud Gateway sert de point d'entrée unique vers les microservices de notre plateforme bancaire. Il reçoit les requêtes des clients et les redirige vers le service concerné selon les routes configurées. Par exemple, une requête de consultation des comptes est dirigée vers Account-Service, tandis qu'une requête de consultation des transactions est dirigée vers Transaction-Service. Les clients peuvent ainsi accéder aux différents services à travers une même adresse.

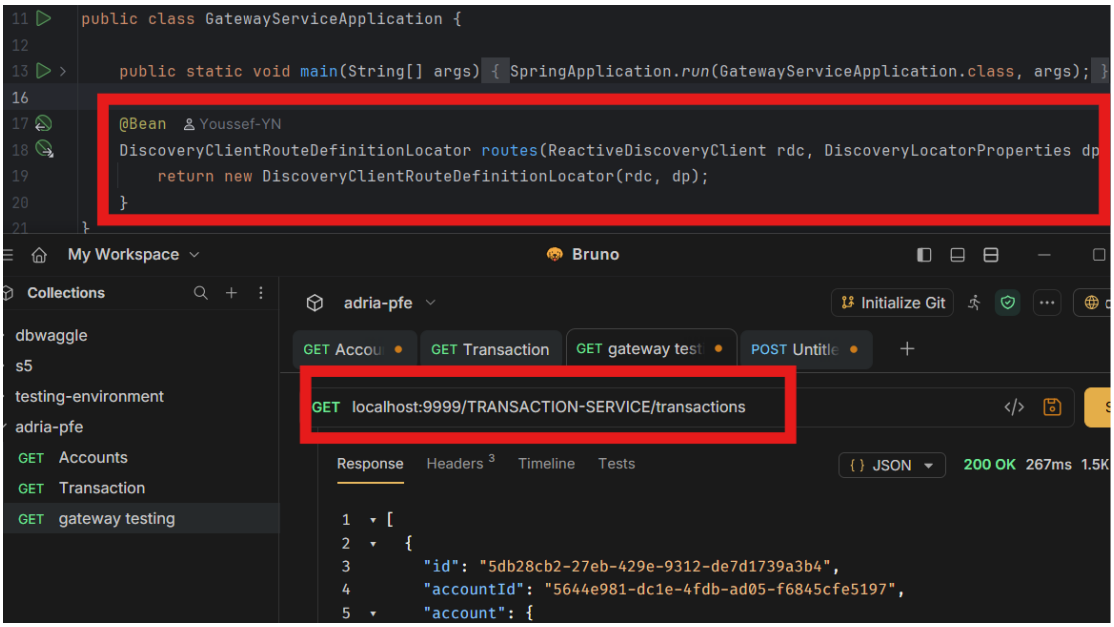


Figure 10: Spring Cloud Gateway

4.2. Eureka Discovery Service

Eureka Discovery Service permet aux microservices de se découvrir sans connaître directement leurs adresses et leurs ports. Chaque service s'enregistre auprès du serveur Eureka avec son nom et ses informations de connexion. Les autres composants peuvent ensuite retrouver ce service à partir de son nom. Dans notre plateforme, ce mécanisme permet notamment à OpenFeign de retrouver Account-Service lors des appels effectués depuis Transaction-Service.

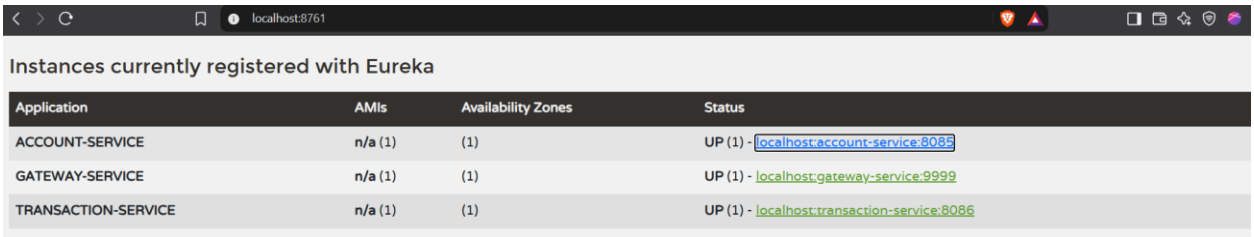


Figure 11: Eureka dashboard

4.3. Config Service

Le service de configuration utilise Spring Cloud Config en mode natif pour centraliser le paramètre bank.transaction.max-amount=1000. Le microservice transaction-service récupère cette valeur au démarrage et vérifie le montant avant l'enregistrement. Toute transaction dépassant ce plafond est refusée avec le statut HTTP 400.

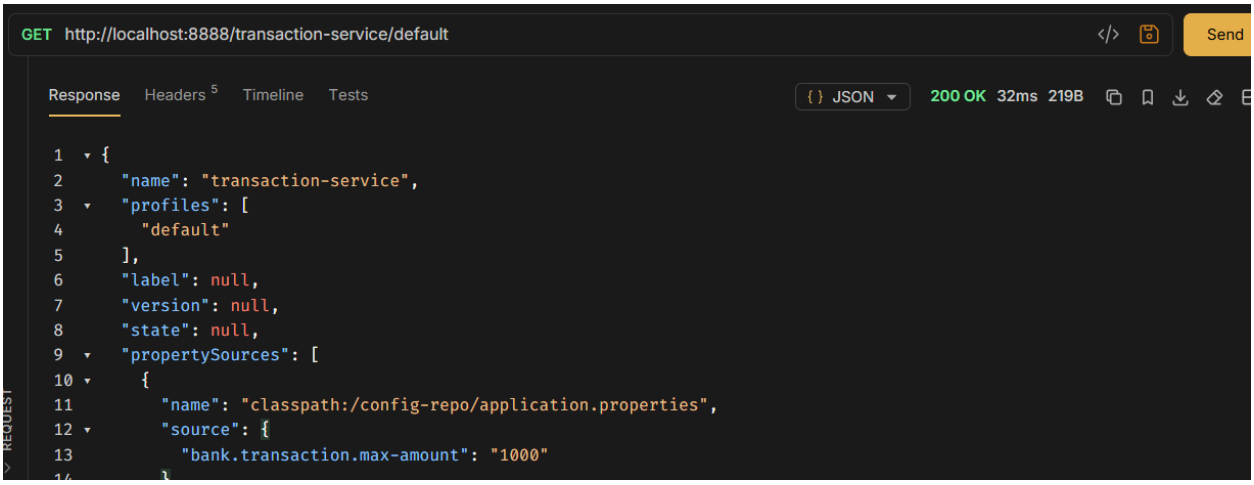


Figure 12: Config Service

5. ChatBot Agentique

Le chatbot utilise Spring AI pour communiquer avec un modèle OpenAI et répondre aux messages des utilisateurs. Le bot Discord, déjà configuré lors d'un précédent projet, sera réutilisé pour proposer une interface de discussion. La clé API OpenAI et le token du bot Discord sont stockés dans des variables d'environnement, puis référencés dans application.properties avec les expressions ${OPENAI_API_KEY} et ${DISCORD_TOKEN}. Cette approche évite d'écrire les clés directement dans le code et réduit le risque de les publier accidentellement sur GitHub.

La création d'un bot Discord se fait depuis le Discord Developer Portal, en créant une application puis en récupérant le token du bot. Ce token permet au service de se connecter à Discord et d'interagir avec les utilisateurs. Dans notre cas, nous réutilisons un bot déjà configuré lors d'un précédent projet.

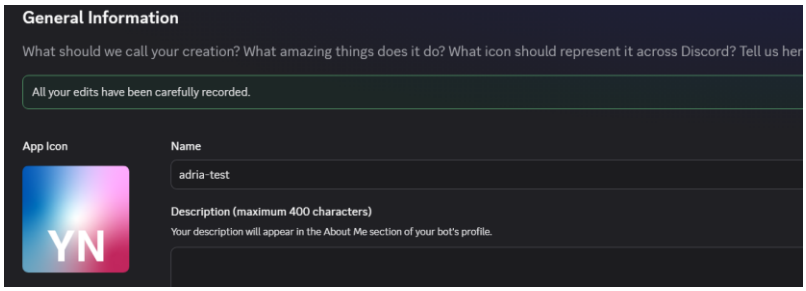


Figure 13: Discord bot

6. Client Flutter

Le client Flutter est organisé en deux parties, les comptes et les transactions. La partie comptes présente les informations principales de chaque compte, comme son identifiant, son statut, sa devise et son solde. La partie transactions affiche les opérations avec leur montant, leur devise et leur date. Cette organisation permet de consulter les données bancaires de manière simple et lisible.

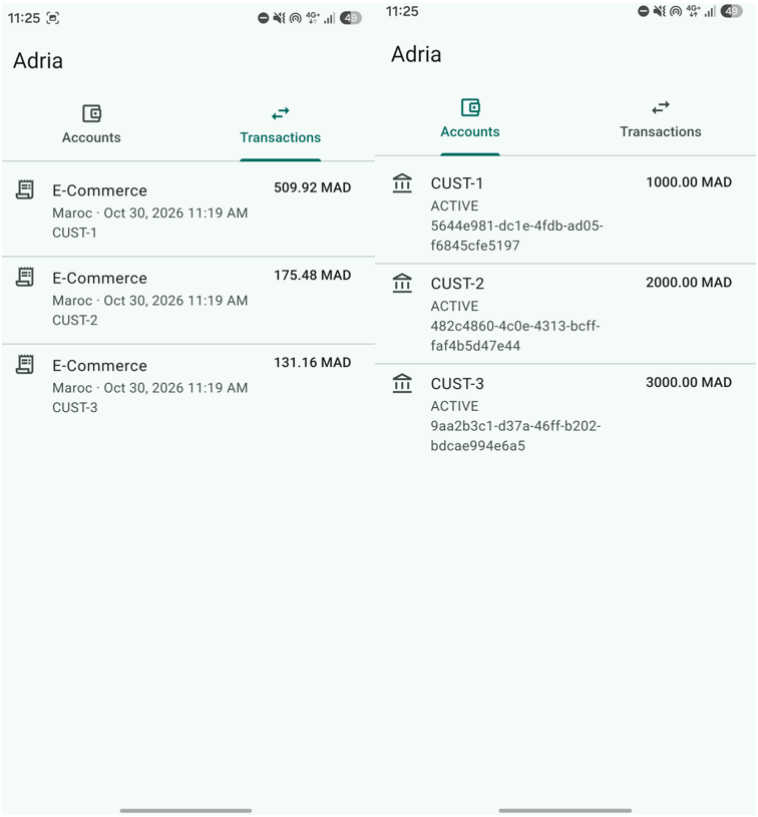


Figure 14: Client Flutter